

IT6206 – Software Quality Assurance — Study Guide & Exam Revision

Exam format: 2-hour paper, **4 structured questions**. Core references: [1] ISTQB Foundation (Rex Black), [2] Concise Guide to SQA (O'Regan), [3] Software Testing & QA (Naik & Tripathy), [4] Mobile Test Automation with Appium (Verma).

How to use this document: The **cheat sheet** below is for last-minute cramming. Sections **1–14** teach each topic in plain language with definitions, examples, and exam tips. The **Quick-recall list** at the end lists the facts most likely to be asked.

Definitions: QA = process/preventive · QC = product/evaluate finished (testing is QC) · Testing = find defects · Debugging = dev removes defect. **QC ⊂ QA**. **Chain: Error** (human mistake) → **Fault/Defect** → **Failure** (fault executed). Galin **9** error causes.

7 Testing Principles: (1) shows presence not absence, (2) exhaustive impossible, (3) early testing saves, (4) defects cluster (Pareto), (5) pesticide paradox, (6) context dependent, (7) absence-of-errors fallacy.

Standards: ISO 9000-3 (**8** QM principles) · CMM/CMMI · Bootstrap (**31** attrs, 3 classes).

Test process — 7 activities: Plan → Monitor&Control → Analysis → Design → Implementation → Execution → Completion. Basis→conditions→cases→procedures→suites→testware. **Traceability = bi-directional** → RTM.

SDLC: Waterfall (late) · **V-model** (Req↔Accept, Spec↔System, Arch↔Integration, Detail↔Unit) · Iterative. **Methods:** White-box (code known) · Black-box (unknown) · Grey-box (partial). **Static = verification** (no run) · **Dynamic = validation** (run).

Black-box techniques: EP (1 val/partition) · BVA (edges: 0,1,max,max+1) · Decision table (2ⁿ rules) · State transition (**States/Transitions/Events/Actions**) · Use case. **White-box:** Statement vs Decision coverage — **decision ≥ statement**. Coverage = exercised/total × 100. **Experience:** Error guessing · Exploratory (charter, time-box) · Checklist.

Static reviews (formality 1): Informal < Walkthrough (author-led) < Technical (moderator, consensus) < **Inspection** (most formal, reader, root cause + process improvement).

Test levels: Component/Unit (devs) · Integration (interfaces; **Stub** = called by / **Driver** = calls) · System (independent team) · Acceptance (**UAT/OAT/CAT/RAT/Alpha/Beta**). **Non-functional:** performance, load, stress, security (**C-I-A**), scalability, reliability (**MTBF = MTTF + MTTR**), documentation.

STLC — 6 phases: Req Analysis → Test Planning → Case Design → Env Setup → Execution → Closure. **Priority = QA (order to fix)** · **Severity = Dev (impact)**. **QC:** 5 metric categories · PCE · Crosby COPQ · **IEEE 829** summary report.

XPath: Absolute `/` (fragile) · Relative `//` (preferred) · `//tag[@attr='val']` · `contains()/starts-with()/text()` · axes (following/ancestor/child/parent/self). **Selenium — 4 components:** WebDriver · IDE (record/playback) · Grid (**Hub-Nodes**) · RC (**deprecated**). Waits: implicit/explicit/fluent. Alerts: simple/prompt/confirmation.

CI/CD: CI = auto build+test on commit · CD = auto deploy validated builds. **Jenkins** (Java, port 8080, Master-Slave, **Jenkinsfile** Declarative vs Scripted, stages Build/Test/Deploy). **Git** = distributed VCS (Torvalds 2005): init/add/commit/branch/checkout/merge.

Appium: Node.js server · **JSON over HTTP** · session ID · **UIAutomator (Android) / XCUITest (iOS)** · no app recompile · cross-platform. Apps: **Native / Browser / Hybrid**. Mobile arch 3 layers: Presentation/Business/Data.

Test Reporting: report = summary of activities + final results. **IEEE 829**. Frameworks: Linear/Module/Library/BDD/Data-driven/Keyword-driven/Hybrid.

What is software, and what is "quality"?

Software is more than code. The IEEE/ISO definition names **4 components:** computer **programs, procedures, documentation, and data**. Quality problems can hide in any of these — a correct program with a wrong user manual is still a low-quality product.

Software Quality has two IEEE definitions, and both matter: 1. The degree to which a system meets its **specified requirements** (did we build it *right*?). 2. The degree to which it meets **customer/user needs & expectations** (did we build the *right thing*?).

A system can pass every written requirement and still fail definition 2 if the requirements themselves missed what the user actually needed.

QA vs QC vs Testing vs Debugging (very common exam question)

Students confuse these constantly. The key idea: **QA is about the process, QC is about the product.**

Concept	Focus	What it does
Quality Assurance (QA)	<i>Process, preventive</i>	Detects and prevents the causes of errors early. Part of quality management. Reduces the cost of quality.
Quality Control (QC)	<i>Product</i>	Set of activities that evaluate a finished product and withhold ones that don't qualify. Testing is a QC activity.
Testing	Detecting	Finds defects & failures. Supports QC — but testing alone is <i>not</i> enough for quality.
Debugging	Fixing	A development activity: the developer finds, analyses, and removes a defect.

Remember: QC is a subset of QA. QA builds a good process; that process includes good testing (QC); testing feeds debugging.

What is SQA?

"A systematic, planned set of actions to provide adequate confidence that the software development/maintenance process conforms to functional requirements AND managerial requirements (schedule + budget)."

Note the two halves: SQA cares not only that the software works, but that it was built **on time and on budget**.

Why do we test? (Objectives)

- Verify all specified requirements are met.
- **Find failures and defects** (the primary focus).
- Give stakeholders information to make **informed decisions** (e.g., ship or delay).
- **Reduce the risk** of inadequate quality reaching users.
- Comply with contractual, legal, or regulatory standards.

Error → Fault → Failure (know this chain cold)

This is the vocabulary of the whole course: - **Error** — a *human mistake* (grammatical or logical) made by a developer. - **Fault / Defect** — the error, now baked into the system, causing improper functioning. - **Failure** — what happens when that fault is **executed under the right conditions** → the user experiences unexpected behaviour.

Not every error becomes a fault, and not every fault becomes a failure. **Galin's example:** 17 errors produced only 8 faults, which caused only 3 failures. A fault in code that never runs never becomes a failure.

Root cause = the earliest action/condition that caused a defect — usually an **organizational** issue (e.g., poor communication). *Root cause analysis* traces failures back to prevent recurrence, not just patch symptoms.

Galin's 9 categories of software error causes

1. Faulty requirements definition
2. Client–developer communication failures
3. Deliberate deviations from requirements
4. Logical design errors
5. Coding errors
6. Non-compliance with documentation/coding instructions
7. Shortcomings of the testing process
8. Procedure errors
9. Documentation errors

Manual vs Automation testing

- **Manual:** a human executes tests step-by-step; no scripts.
- **Automation:** tools/scripts run the tests → cost savings, shorter durations, better accuracy and efficiency.
- **Automation advantages:** accuracy, better documentation, comprehensive info, less manpower during execution, shorter testing periods.
- **Automation disadvantages:** high tool purchase/training cost, high script-development cost, high manpower for test preparation, and some areas remain uncovered.

The Seven Testing Principles (MEMORIZE — almost guaranteed exam question)

Each principle corrects a common misconception: 1. **Testing shows the presence of defects, not their absence.** Passing tests never *prove* software is bug-free. 2. **Exhaustive testing is impossible.** You can't test every input combination — use **risk analysis + priorities** instead. 3. **Early testing saves time and money** ("shift-left"). A defect found in requirements is far cheaper than one found in production. 4. **Defects cluster together.** A small number of modules usually hold most defects (**Pareto principle**) — focus effort there. 5. **Beware the pesticide paradox.** Running the same tests repeatedly stops finding new bugs — **vary your tests and data**. 6. **Testing is context dependent.** You test a banking app differently

from a game. 7. **Absence-of-errors is a fallacy.** Finding and fixing many defects still doesn't guarantee a useful, successful system if it's the wrong system.

SQA models & standards

- **ISO 9000-3** — guidelines for applying ISO 9001 to software. Built on **8 quality-management principles**: Customer focus, Leadership, Involvement of people, Process approach, System approach to management, Continual improvement, Factual approach to decision-making, Mutually-supportive supplier relationships. **Certification path**: (1) develop an SQA system, (2) implement it, (3) undergo certification audits.
- **CMM / CMMI** — Carnegie Mellon (1986). Focuses on **maturing the software process** through defined levels. CMMI integrates agile principles.
- **Bootstrap Model** — measures maturity via **31 quality attributes** grouped into 3 classes: **process, organization, technology**.

The big idea

Testing ≠ just executing tests. Execution is only one activity. There is **no one-size-fits-all** process — it's shaped by the SDLC model, test levels/types, product & project risks, business domain, operational constraints, organizational policies, and required standards.

The 7 test activities (may overlap or iterate)

Planning → Monitoring & Control → Analysis → Design → Implementation → Execution → Completion.

Activity	What happens	Simple question it answers
Test Planning	Define objectives & approach; choose techniques, tasks, schedule.	"What's our strategy?"
Monitoring & Control	Compare status against plan; report to stakeholders. Uses exit criteria (the "definition of done"). Control = corrective action.	"Are we on track?"
Test Analysis	Analyse the test basis to identify test conditions . Also finds defects in the basis (ambiguities, omissions, contradictions).	"What should we test?"
Test Design	Turn conditions into test cases (inputs + expected results).	"How do we test it?"
Test Implementation	Build testware : procedures, suites, test data, environment.	"Do we have everything to run?"
Test Execution	Run suites; compare actual vs expected; log pass/fail/blocked; report defects.	"Does it work?"
Test Completion	Close defects, write the test summary report , archive testware, capture lessons learned.	"What did we learn?"

Key vocabulary (build up in this order)

- **Test basis** = the body of knowledge tests are based on (requirements, design, the component, risk report).
- **Test condition** = a testable aspect of the test basis.
- **Test case** = specific inputs + expected results.
- **Test procedure** = a sequence of test cases plus setup/wrap-up.
- **Test suite** = a set of test cases/procedures for a cycle.
- **Testware** = all work products the test process produces.

Traceability & the RTM

Traceability links two or more work products, and it must be **bi-directional** (forward: requirement → test; backward: test → requirement). Benefits: **impact analysis** of changes, auditability, **coverage measurement**, IT governance, coherent reports, and relating testing to stakeholders. This linkage is the basis of the **Requirement Traceability Matrix (RTM)**.

SDLC models and how testing fits

- **Waterfall (sequential)**: testing happens at the *end* → defects are found late and are expensive; weak feedback loops.
- **V-model**: every development phase has a **matching test phase**, so test design starts early. The 4 levels pair up: **Requirements ↔ Acceptance, Specification ↔ System, Architectural design ↔ Integration, Detailed design ↔ Unit**.
- **Iterative/Incremental** (RUP, Scrum, Kanban, Spiral, Agile, XP): builds in small increments. Trade-offs: more **regression testing**, defects that fall outside an iteration's scope, and sometimes less thorough testing.
- **Good testing in any model**: every dev activity has a matching test activity; each level has its own objectives; test design begins *during* the matching dev activity; testers are involved early (e.g., in reviews).

Testing methodologies (how much of the code do we see?)

	White-box	Black-box	Grey-box
Internal code	Known	Unknown	Partially known
Also called	Structural / Glass-box / Clear-box	Behavioural	—
Based on	Internal structure, code	Requirements/specs, inputs/outputs	A mix of both

White-box coverage types: **Statement, Decision, Branch, Condition, Multiple-condition, Finite-state-machine, Path.**

Functional vs Non-functional vs Change-related testing

- **Functional** = *what* the system does (compliance with functional requirements). Two perspectives: **requirement-based** and **business-process-based**. Done at all levels.
- **Non-functional** = *how well* it works (performance, load, stress, usability, portability, security). Done at all levels.
- **Change-related:**
- **Confirmation testing (re-testing):** after a fix, confirm the original failure is gone.
- **Regression testing:** after *any* change, confirm no *new* defects appeared in unchanged areas.

Static vs Dynamic testing (common exam table)

Static	Dynamic
No code execution	Code is executed
Prevents defects	Finds & fixes defects
Verification ("building it right?")	Validation ("building the right thing?")
Done before compilation	Done after compilation
Reviews, static analysis	Unit/Integration/System testing; BVA, EP

Techniques fall into **three complementary categories** — each is good at catching a different *kind* of defect, so real projects use all three.

Black-box techniques

Derived from the **test basis** (requirements, use cases, user stories); coverage = the items in that basis covered. You don't look at the code — only inputs and expected outputs.

1. **Equivalence Partitioning (EP):** divide inputs into groups ("partitions") the system should treat the same, then test **one value per partition** (both valid and invalid ones). *Bank interest example:* rates 3% (\$0–100), 5% (\$100–1000), 7% (\$1000+) → 3 valid partitions + 1 invalid (negative amounts). One test per partition instead of hundreds of values.
2. **Boundary Value Analysis (BVA):** defects cluster at the **edges** of partitions, so test the boundaries. *Printer accepts 1–99 copies* → test **0, 1, 99, 100**. *Text field 1–30 chars* → test **0, 1, 30, 31**. Pairs naturally with EP.
3. **Decision Table Testing:** captures **combinations of conditions** → **actions** (cause-effect). With n conditions you get **2ⁿ rules**. Great for business rules; also exposes gaps/ambiguities in the spec. When combinations explode, **prioritize** the important ones.
4. **State Transition Testing:** for **finite-state machines**, where the *same* input gives a *different* output depending on the current state. It has **4 parts: States, Transitions, Events, Actions**. A **state diagram** shows valid transitions; a **state table** also captures the invalid/negative ones. *ATM PIN example:* 7 states, 4 events (e.g., wrong PIN three times → card locked).
5. **Use Case Testing:** tests a whole transaction from start to finish (actor–system interaction). Covers the **main success scenario** plus **extensions** (alternate/exception branches), with pre- and post-conditions. It's the natural basis for **system and acceptance** tests.

White-box techniques

Derived from the **code/structure**; coverage = structural items exercised. **Coverage = (items exercised / total items) × 100%.**

1. **Statement Testing/Coverage:** execute every statement at least once. This can miss branches — it only guarantees the "true" path of a decision is hit.
2. **Decision Testing/Coverage:** exercise **both the True and False** outcome of every decision (IF, loops, CASE).

Key relationship (exam favourite): 100% decision coverage *guarantees* 100% statement coverage — **but not the reverse**. Decision testing is the stronger criterion.

Experience-based techniques

Rely on the tester's skill and intuition. They **complement** the systematic techniques and are especially useful when specifications are poor or missing. 1. **Error Guessing**: the tester anticipates likely mistakes — division by zero, blank input, empty files, wrong data type. The structured form uses **defect/failure lists** from past projects. 2. **Exploratory Testing**: minimum planning, maximum execution. Guided by a **test charter** (time-boxed, ~1–2 hours); design and execution happen **in parallel** while the tester learns the system. Best when specs are poor and time is short. 3. **Checklist-based Testing**: tests are driven by a **checklist** of conditions — giving consistency and guidance and higher coverage, but with **less repeatability** (two testers may interpret items differently).

What it is and why it's cheap

Static testing examines work products **without executing** them. Because it starts early in the life cycle, it removes defects when they are cheapest to fix (per principle 3, *early testing saves*).

Defects it finds easily: deviations from standards, missing requirements, design defects, non-maintainable code, and inconsistent interface specifications.

Two branches of static methods

- **Reviews** — people examine the work product (Formal or Informal).
- **Static Analysis** — tools examine code without running it: finds unused variables, dead code, infinite loops, variables with undefined values, wrong syntax; also computes **data flow, control flow, and cyclomatic complexity**. Tools: SourceMeter, PyCharm, PMD, FindBugs.

The four review types (know this comparison — very likely exam)

Formality increases down the table, and so does the amount of documentation and process.

Review	Formality	Led by	Preparation	Scribe	Main purpose
Informal (buddy check, pairing)	Least	—	Optional	—	Cheap defect detection; common in Agile; not documented
Walkthrough	Low–medium	Author	Optional	Mandatory	Find defects + knowledge transfer; scenarios/dry-runs; good for high-level docs
Technical Review	Medium–high	Trained moderator (not the author)	Required	Mandatory (not the author)	Consensus among technical experts + find defects
Inspection	Most formal	Moderator	Required	Mandatory	Defects + process improvement + root-cause analysis ; uses rules & checklists; has a dedicated reader ; formal documented output

Benefits of static testing

Lower cost (less rework), reduced development time, fewer defects slipping into dynamic testing, more maintainable code, better insight into quality, faster feedback with tools, and more team communication.

Each level answers a different question and has its own **objectives, test basis, test objects, typical defects, responsibilities, and environment**. Think of them as zooming out: from a single function to the whole business process.

Functional levels (the V-model levels)

Level	Focus	Test basis	Typical defects	Who does it
Component / Unit	Individual, separately-testable pieces	detailed design, code, data model, component spec	wrong functionality, data-flow, incorrect logic	Developers
Integration	Interfaces & interactions between pieces	design, sequence diagrams, interface/protocol specs, use cases, architecture	interface mismatch, data/timing/comm failures, wrong assumptions	Component-integ: devs; System-integ: testers
System	Whole-system behaviour (functional + non-functional)	system/SW requirement spec, risk report, use cases, state diagrams, manuals, epics/stories	wrong calculations, wrong data/control flow, failed end-to-end tasks	Independent test team
(User) Acceptance	Real user needs & business processes	business processes, user/business & system requirements, contracts, regulations	workflows don't meet needs, wrong business rules, contract/regulatory gaps	Users / customers

Stubs and Drivers (used in unit/integration testing to stand in for missing software): - **Stub** = something **called by** the component under test (a fake "below" it). - **Driver** = something that **calls** the component under test (a fake "above" it). - Mock objects serve a similar substitution role.

Integration testing splits into **Component integration** (right after unit testing; automated; done by devs) and **System integration** (systems, microservices, web services; done by testers). **System testing** is usually the final test done by the development organization and covers **both verification and validation**.

Forms of acceptance testing

- **UAT** — by the intended users, in a real or simulated environment.
- **OAT (Operational)** — by operations/sysadmin staff (backup, recovery, security).
- **Contractual (CAT)** — checks the software against a contract.
- **Regulatory (RAT)** — checks it against laws/policies/regulations.
- **Alpha** — at the developer's site, by people outside the dev team.
- **Beta** — at the customer's/external site, by outside users.

Non-functional testing (measurable quality attributes, usually prioritized)

Type	Purpose
Interoperability	Correct end-to-end interaction with other systems (IoT, web services, COTS).
Performance	Speed, response time, stability, resource usage; locate bottlenecks.
Scalability	Behaviour as workload grows toward engineering limits; user limits, degradation.
Stress	Behaviour under extreme/heavy load; robustness & error handling; find memory leaks; push load until failure.
Security	Confidentiality, Integrity, Availability (C-I-A) ; guard against unauthorized access, eavesdropping, DoS; authentication, virus checks.
Load & Stability	Stays stable under full load for a long period (simulated virtual users).
Reliability	Stays operational long-term. MTTF (mean time to failure), MTTR (mean time to repair), MTBF = MTTF + MTTR .
Regression	Confirms changes introduced no new defects; picks a subset of tests to maximise defect-finding at minimum cost.
Documentation	Accuracy/readability of manuals. Three levels: Read test, Hands-on test, Functional test .

What STLC is

Where the SDLC covers building the *whole* product, the **STLC** is the **testing-only** sub-process inside it — a defined sequence of phases the test team follows.

6 phases (memorize the order): Requirement Analysis → Test Planning → Test Case Design & Development → Test Environment Setup → Test Execution → Test Closure.

Every phase is guarded by criteria: - **Entry criteria** = conditions/documents needed to **begin** a phase. - **Exit criteria** = items that must be complete before you can **end** it.

What happens in each phase

- **Requirement Analysis** — **3 activities**: (1) **Scope definition**, (2) **RTM (Requirement Traceability Matrix)** preparation, (3) **Automation analysis** (deciding what can be automated).
- **Test Planning**: prepared by the **QA Team Lead / Manager**. It answers **WHAT to test** and **WHAT RESOURCES** are needed.
- **Test Case Development** — **3 activities**: Test Scenario Identification → Test Case Writing → Test Data Preparation.
- **Test Environment Setup** — **3 steps**: Design → Setup → **Smoke Testing** (a quick check that the build is stable enough to test properly).
- **Test Execution**: run tests and manage defects (detailed below).
- **Test Closure**: wrap up and report.

The Defect (Bug) Life Cycle

A defect moves through states from discovery to closure: **New** → **Assigned** → **Active** → **Test/Fixed/Ready for Retest** → **Verified** → **Closed**, plus the side states **Reopened** (retest failed), **Deferred** (fix postponed), and **Rejected** (not a valid defect).

Priority vs Severity (very common exam question)

Both rank defects, but from different viewpoints:

	Priority	Severity
Viewpoint	QA — the <i>order</i> in which to resolve	Dev — the <i>impact</i> + code complexity
Levels	Low / Medium / High / Urgent	Critical (Severity 1) / Major (Sev 2) / Medium (Sev 3) / Low (Sev 4)

Example: a company logo spelled wrong is **low severity** (doesn't break anything) but **high priority** (embarrassing, must fix before launch).

Test Execution & Closure details

- **Execution activities:** **System Integration Testing (SIT, black-box)** → **Defect reporting** → **Defect mapping** (back to the RTM) → **Re-testing** → **Regression testing**.
- **Test Closure** produces the **Test Completion Report**, which contains: Test Summary Report Identifier, Summary, Variances, Summary Results, Evaluation, **Planned vs Actual Efforts**, and **Sign-off**.

Quality Control is where you *measure and watch* the testing effort and act on what you see.

7.1 Test Monitoring & Control

The **test manager** is responsible. The purpose is to give **feedback and visibility** into progress and to trigger **corrective action** when things drift. Objectives include: monitoring the plan/schedule and key parameters, milestone reviews, tracking risks and issues/change requests, driving corrective actions to closure, managing resources, and **reporting status**.

7.2 Metrics used in testing — 5 sample categories

1. **Customer satisfaction.**
2. **Project management of testing** — schedule accuracy, effort estimation.
3. **Execution of testing** — defects/CRs/issues, **age of open defects**, problem arrivals, **PCE (Phase Containment Effectiveness)** — how many defects were caught in the same phase they were introduced), test progress, cumulative arrivals.
4. **Customer care** — query arrival/closure, outages, availability.
5. **Cost of quality** — the **CMMI internal maturity score (1–10; a score ≥ 7 is "satisfied")** and **Crosby's Cost of Poor Quality (COPQ)** (what defects cost the business).

7.3 Test Reports

- **IEEE 829** is the standard for the **test summary report**.
- **Contents:** summary of activities/results, completed deliverables, new risks/issues, schedule/effort/budget status, test status, key risks/issues, milestone status, and planned activities.
- **Roles:** the **Test manager** presents status; **Management** provides support; the **Project manager** issues a recovery/exception report when needed.

Sub-topics: 8.1 Intro · 8.2 Choosing the Right Tool · 8.3 Effective Use · 8.4 Environments · 8.5 Manual vs Automation · 8.6 Benefits & Risks · 8.7 Tools.

What & why

Test automation is using software tools to **develop and execute** tests — the tool enters test data, compares expected vs actual results, and generates reports. **Why automate?** To improve accuracy, quality, and coverage, and to make execution faster.

When (and what) to automate

Automation pays off for repetitive, stable, high-volume work: - **Regression testing** (once the app is fairly stable). - **Smoke testing** (a quick go/no-go check on a new build). - **Static & repetitive tests** (unchanging from cycle to cycle). - **Data-driven testing** (e.g., login or search validated against many data sets). - **Load & performance testing** (no realistic manual alternative exists).

Which tests to automate: business-critical cases, smoke tests, tests that are hard/tedious to do manually (complex calculations), and modules that are **stable enough** — automating an unstable module wastes effort.

Types: **Functional** automated tests (tools: **Selenium, UFT, RFT, VSTS**) vs **Non-functional** automated tests (tools: **JMeter, LoadRunner**).

8.2 Choosing the right tool — 4 steps

1. **Identify the requirements** (project, scope, kind of testing, needed features).
2. **Evaluate tools and vendors** — shortlist the best fits.
3. **Estimate cost and benefit** for the shortlist.
4. **Make the final decision** — ideally trial the tool in your real work setting before buying. *Key factors:* platform compatibility, user-friendliness / keyword-test support, record-and-playback + checkpoints, maintainable & reusable

tests, integration with your CI/CD and defect/source-control ecosystem, and support for enterprise apps (SAP, Oracle, Salesforce, Workday).

8.3 Using tools effectively

Train the team → the **automation architect** builds a long-term **framework** (types: **linear**, **modular**, **data-driven**, **keyword-driven**, **hybrid**) so scripts need minimal maintenance when the app changes → create an **execution plan** (which OS/browser/hardware) → write scripts (naming conventions, kept in source control) → generate reports → **maintain** the scripts. A poor framework makes maintenance a *nightmare* — the main reason many automation projects fail.

8.4 Test environment (test bed)

A **test environment/test bed** is the combination of **software + hardware** the team uses for testing (sometimes including test data). Setting one up involves: **(1) Test server, (2) Network, (3) Test PC, (4) Bug-reporting tools, (5) Test data.** When copying production data, mask **PII (Personally Identifiable Information)**. Two anonymization approaches: **BlackList** (change only the listed fields) vs **WhiteList** (anonymize everything *except* the listed fields).

8.5 Manual vs Automation

Criterion	Manual	Automation
Accuracy	Human error possible	Higher (no human error)
Testing at scale	Slow	Fast
Cost efficiency	Costlier (experts)	Cheaper once set up
User experience	Better (human observation)	Machines lack cognition
Best for	Exploratory, Usability, Ad-hoc	Regression, Load, Performance, Repeated execution

8.6 Benefits & Risks

- **Advantages:** Speed, Reliable, Repeatable, better Coverage, Reusable across app versions.
- **Risks:** unrealistic expectations of the tool, overlooking what human testers do better, wrong time/effort estimates, tool-environment incompatibility, budget issues, and **vendor issues** (no support/updates, vendor liquidation, or a free tool becoming licensed).

8.7 Tools at a glance

- **Licensed:** Appium, Tellurium, QTP, TestComplete, WinRunner, Rational Functional Tester, VSTS, SilkTest.
- **Open source:** Testim, Selenium, JMeter, OpenSTA, SoapUI.

Why XPath exists

When Selenium can't find a web element using simple locators (id, class, name), **XPath** navigates the page's HTML **DOM** to pinpoint it. XPath is a small query language for XML/HTML. Basic format:

```
Xpath = //tagname[@Attribute='Value']
```

The available locators are: **ID, Classname, Name, Link text, XPath, CSS path.**

Absolute vs Relative XPath (know the difference)

- **Absolute XPath** starts from the root with a **single slash /** and lists every node down to the target — e.g. `/html/body/div[1]/section/.../h4[1]/b`. It's direct but **brittle**: any change to the page structure breaks it.
- **Relative XPath** starts anywhere in the DOM with a **double slash //** — e.g. `//*[@class='featured-box']//*[text()='Testing']`. Shorter and far more robust — **preferred in practice.**

Finding elements by attribute (the workhorse patterns)

1. **Basic:** `//input[@type='text']`, `//label[@id='message23']`, `//a[@href='...']`.
2. **contains()** — for values that change or when you only know part: `//*[contains(@name,'btn')]` matches `btnLogin`, `btnReset`, etc.
3. **OR / AND** — combine conditions: `//*[@type='submit' or @name='btnReset']`; `//input[@type='submit' and @name='btnLogin']`.
4. **starts-with()** — for attributes whose start text is stable but end changes dynamically: `//label[starts-with(@id,'message')]`.
5. **text()** — match by visible text: `//td[text()='UserID']`.

XPath axes (navigating relative to a node)

Axes find elements *related to* the current node — essential for dynamic pages: **following**, **ancestor**, **child**, **preceding**, **following-sibling**, **parent**, **self**, **descendant**. Examples: `//*[@type='text']//following::input[1]` (next input after a

text field), `//*[@id='rt-feature']/parent::div[1]` (its parent div), `//*[@type='password']/self::input` (the node itself — always exactly one match).

Sub-topics: 10.1 Intro · 10.2 Architectures · 10.3 Recorders · 10.4 Selenium API.

Types of automated testing for web apps

- **Unit testing** — individual functions/components in isolation; runs as part of CI/the build.
- **Integration testing** — how the front-end talks to the back-end/DB, and how microservices talk to each other.
- **End-to-end testing** — the whole workflow, simulating real user interactions (clicks, navigation).

Popular suites include **Selenium** (the focus), **Cypress** (JS, end-to-end), **TestCafe** (JS, cross-browser/parallel), and **Puppeteer** (Node.js, headless).

The Selenium suite — 4 components

1. **Selenium WebDriver** — language-specific bindings that drive a real browser; used to build robust regression suites; provides drivers for Chrome, Firefox, Safari, etc.
 2. **Selenium IDE** — a Chrome/Firefox/Edge add-on for **record-and-playback**; lets people with little/no programming create tests and quick bug-repro scripts.
 3. **Selenium Grid** — runs tests **in parallel across many machines** using a **Hub-and-Nodes** architecture (the hub is the central server; nodes run the actual tests against various browser/OS combos).
 4. **Selenium Remote Controller (RC)** — **deprecated**; no longer a component. It used to inject JavaScript and needed a separate server.
- **Advantages:** cross-browser, multi-language, record & playback, integrates with test-management and CI tools, large community.
 - **Limitations:** only supported browsers; weak mobile support; needs programming knowledge; web-only (not desktop); poor at recognising images / non-HTML-CSS content.

10.2 Architectures of automated testing suites

- **Record-and-Playback** — simple, but brittle on complex apps.
- **Keyword-driven** — build tests from predefined keywords; flexible but needs upfront design.
- **Data-driven** — one test driven by many input rows from a file/DB.
- **Modular** — break tests into small independent components, then combine them.
- **Page Object Model (POM)** — each page is represented by an object holding its elements; makes complex suites easy to maintain.

The three layers of a typical suite: the **Test layer** (the test cases), the **Automation layer** (executes them and drives the UI/DB via libraries/APIs), and the **Framework layer** (test-data management, configuration, and reporting).

10.3 Recorders

Selenium IDE (browser plugin that records using Selenium commands) and **Katalon Recorder** (Selenium-IDE-compatible; can record, debug, execute, and **export to C#/Java/Python/Ruby**). **Limitation:** recordings don't capture everything and often need customization — a recorder is **not a substitute** for manual or code-based testing.

10.4 Working with the Selenium API (WebDriver)

- **Locating elements:** `find_element_by_id` / `_name` / `_class_name` / `_tag_name` / `_link_text` / `_partial_link_text` / `_xpath`.
 - **Interacting** via the `Actions` class — **mouse:** `doubleClick()`, `clickAndHold()`, `dragAndDrop()`, `moveToElement()`, `contextClick()`; **keyboard:** `sendKeys()`, `keyUp()`, `keyDown()`.
 - **Windows & frames:** a **window handle** is a unique string ID used to switch between windows/frames.
 - **Executing JavaScript:** the `JavaScriptExecutor` interface — `executeScript()` (synchronous) and `executeAsyncScript()` (asynchronous).
 - **Wait commands (3 types)** — needed because page elements load at different times, otherwise you get `ElementNotVisibleException`:
 - **Implicit wait** — wait a fixed time before throwing.
 - **Explicit wait** — wait until a specific condition is met.
 - **Fluent wait** — poll at regular intervals until found or timeout.
 - **Alerts (3 types): Simple, Prompt, Confirmation** (plus pop-up windows).
 - **Best practices:** consistent coding style, modular code, use debugging tools.
-

There is **no Lesson 11 slide deck** in the materials — the numbering jumps from **10 (Selenium suites)** straight to **12 (CI/CD)**. If your paper references a "Lesson 11" (commonly **TestNG / Java-based automation**), use general knowledge and verify against your LMS: - **TestNG** = a Java test framework (successor to JUnit), used with Eclipse + Selenium

WebDriver; configured through a `testng.xml` file (groups, order, suites, parameters, parallel runs). - **Key annotations:** `@Test`; `@BeforeMethod` / `@AfterMethod`; `@BeforeClass` / `@AfterClass`; `@BeforeSuite` / `@AfterSuite`; `@BeforeTest` / `@AfterTest`; `@BeforeGroups` / `@AfterGroups`. - (Not sourced from a course PDF — treat as supplementary.)

Sub-topics: 12.1 Intro · 12.2 Setting up Jenkins · 12.3 Jenkins Pipeline · 12.4 Version Control Systems.

The core idea

- **Continuous Integration (CI):** continuously **verify the state of the codebase through automated testing**. It works best when integrated with version control — every commit is checked automatically.
- **Continuous Delivery/Deployment (CD):** regularly **deploy the artifacts that pass CI**, so you stay confident about releasing. (Continuous *Delivery* = ready to release at any time; continuous *Deployment* = automatically pushed to production.)

CI, continuous delivery, and continuous deployment share the same goal — faster, more robust releases — and differ only in the **scope of automation** applied.

The phases

- **CI phase:** code is pushed to the VCS → the build server checks it immediately → developers get **feedback (pass/fail)** → code is stored in the repository.
- **CD phase:** ensures reliable releases; deployments are quick and frequent; deployment is automated.
- **CI & CD pipeline:** *Git repo* → **CI pipeline** (Build → Unit test → Package → Integration test) → *Artifact repo* → **CD pipeline** (deploy to TEST → acceptance tests → STAGING → performance tests → end-to-end tests → deploy to PROD).

Signs of a good CI / CD

- **Good CI: Decoupled stages** (each step does one thing), **Repeatable** (consistent for local and remote devs), **Fail fast** (stop at the first sign of trouble).
- **Good CD:** designed with the whole system in mind (App, Infrastructure, Config, Data), **pipelines** that build confidence as you approach production, and **globally unique versions** so you always know the system's state.
- **Who should know CI/CD:** Developers, QAs, System Administrators.

Jenkins

Jenkins is a widely used open-source CI/CD tool, written in **Java**. It can build projects manually, periodically, or automatically, and has **hundreds of plugins** (Git, Maven, SonarQube, Selenium, JUnit, Cucumber...). - **Master-Slave architecture:** the **Master** schedules and dispatches build jobs, monitors slaves, and records results (it can also build directly); each **Slave** executes the jobs it's given. This is how Jenkins covers many browser/OS combinations without doing it all on one machine. - **Prerequisites:** JDK 1.7+, 2 GB RAM (recommended), cross-OS. Run `java -jar jenkins.war`; by default it serves on **port 8080** via an embedded server called **Winstone**.

Jenkins Pipeline

A **Pipeline** is a set of plugins for building delivery pipelines, written in a **DSL** and stored in a **Jenkinsfile** ("Pipeline-as-code" — versioned and reviewed like any other code). Benefits: auto-builds all branches/PRs, code review, audit trail, single source of truth. - **Two syntaxes:** **Declarative** (newer, richer, easier to read) vs **Scripted**. - **Jargon:** **Pipeline** (the whole build process), **Node** (a machine that runs the pipeline), **Stage** (a logical group like Build/Test/Deploy), **Step** (a single command, e.g. `sh` runs a shell command). - **Typical stages:** **Build** (`sh 'make'`, then `archiveArtifacts`), **Test** (`junit` captures results — failures mark the build *unstable*), **Deploy** (`when { currentBuild.result == 'SUCCESS' }` then `sh 'make publish'` — only if earlier stages passed).

12.4 Version Control Systems (VCS)

A **VCS** tracks and manages changes to source code over time. - **Features:** backup, recording increments (know which version is live), change history, letting **many developers work on the same code base**, merging changes / handling collisions, answering "who did what", **tagging** (point-in-time marks), and **branching** (maintain release versions while main development continues). - **Centralized VCS** (one main trunk, e.g. CVS, SVN) vs **Distributed VCS** (every developer has a full copy, e.g. Git). - **Jargon:** Repository (repo), Server, Client, Working Copy, Trunk/Main, **Head** (the latest revision). - **Actions:** Add, Check in, Check out, Ignore, Revert, Update/Sync, Diff, **Branch, Merge, Conflict, Resolve, Locking**.

Git

Git is a **distributed** VCS created by **Linus Torvalds in 2005** as open-source software. It supports non-linear development, runs on macOS/Linux/Windows, works with HTTP/FTP/ssh, and handles projects small to large. Git has three areas: **working directory** → **staging area** → **git directory (repository)**. Essential commands:

```
git init          # start a repo
git add *.c       # stage files
git commit -m 'Initial project version'
```

```
git log          # view history
git branch testing
git checkout testing # switch branch
git merge testing  # integrate a branch
```

Sub-topics: 13.1 Intro/architectures · 13.2 Types · 13.3 Test strategy · 13.4 Testing fundamentals · 13.5 Tools · 13.6 Appium architecture · 13.7 Native testing with Appium.

Mobile architecture — 3 layers

A well-architected mobile app has independent, easily-scaled layers: - **Presentation layer** — the **UI** (colours, fonts, alignment) and **UX** (how the user interacts). - **Business layer** — the logic and rules: data validation, exception management, security. - **Data layer** — data safety and maintenance: data access, network communication, routing, storage.

The three types of mobile apps

- **Native** — built for one platform (Android in Java; iOS in Swift/AppCode) using its SDK; tied to that OS; runs **online and offline**; best performance. (Android favours independent layers; iOS often uses **MVC**.)
- **Browser-based (mobile web)** — HTML/CSS/JavaScript accessed through a mobile browser; needs a network; cross-platform and easy to maintain; but **lacks device features** like GPS/camera.
- **Hybrid** — web technologies running inside a **native container**, giving a native feel *and* access to device features, with **one code base** for all OSes (great for wrapping an existing website).

13.3 Test strategy

You can't test every line of code — that's inefficient and costly — so you find a strategy that balances **efficiency, reliability, and precision**. - **Fidelity**: the closer the test environment is to a real device, the better. **High-fidelity** tests run on real devices/emulators; **low-fidelity** tests run on a local JVM. - **Flaky tests** = tests that don't pass 100% of the time even when correct. - **Testable architecture** matters — an untestable one forces bigger, slower, flakier tests. The fix is **decoupling**: split the app into layers (presentation/domain/data), make dependencies easy to replace, and avoid direct framework dependencies in business-logic classes.

13.4 Testing fundamentals

- **Android tests** — by *subject*: Functional, Performance, Accessibility, Compatibility. By *scope*: **Unit, End-to-end, Medium** (integration of two+ units). By *location*: **Instrumented** (run on a real device/emulator) vs **Local/host-side** (run on the dev machine/server).
- **iOS tests** — manual on device (system, UI, security, field), manual on emulator (unit, integration, UI), and automation (regression, **BVT — Build Verification Test**, compatibility, performance).

13.5 Tools & frameworks

Four common ones: **Appium**, SmartBear **TestComplete**, **Katalon**, and **Ranorex** — all support record-and-playback and run on real devices and emulators.

13.6 Appium architecture

Appium is open-source and, at its core, a **web server written in Node.js**. It uses a **client-server** model: 1. The client sends a connection as a **JSON object over HTTP**. 2. The server **creates a session** and returns a **session ID** (kept until the server stops). 3. The server **executes** the commands and **returns the status**. Under the hood it drives the device using **UIAutomator (Android)** and **XCUITest (iOS)**.

Advantages: open-source with an active community; supports many languages (Java, JavaScript, C#, PHP, Python, Perl...); **doesn't require recompiling or modifying the app** (you test the exact build you'll ship); enables cross-platform tests; lets you reuse code across Android and iOS.

13.7 Native testing with Appium

- **Android**: Appium proxies commands to a **UIAutomator** script on the device; `bootstrap.jar` acts as a TCP server sending the commands.
- **iOS**: Appium uses Apple's **UIAutomation** API; commands run via `bootstrap.js`.
- **Prerequisites**: JDK, Android Studio, extra Android SDK tools, the Appium jar, Appium Desktop client, and Eclipse — plus **Appium Doctor** to verify the installation.

Sub-topics: 14.1 Reporting Process · 14.2 Reporting Frameworks · 14.3 Software Tools.

What a test report is (and why it matters)

"A Test Report is a document that contains a summary of all test activities and the final test results of a testing project."

It lets stakeholders judge the tested product's quality and decide whether to **release or postpone** — if the report shows many defects remaining, the release can be delayed. A **Test Summary Report** is the *consolidated* view produced after a cycle, unlike **Daily Status Reports** which only give the day's results. Leave enough time between submitting the report and shipping.

What to include in a Test Summary Report

Keep it concise and relevant: **Test Objective**, **Areas Covered**, **Areas Not Covered**, **Testing Approach**, **Defect Report**, **Platform Details**, and an **Overall Summary**.

The 10 steps to create one

1. Capture the **purpose** of the document.
2. Give a product **overview**.
3. Capture the **testing scope** (in/out of scope, and what wasn't tested and why).
4. Capture the **metrics** — # test cases planned / executed / passed / failed.
5. Capture the **types of testing** performed.
6. Capture the **test environment and tools** (app URL, DB version, bug-tracking tool).
7. Record **learnings** (critical issues and how they were solved).
8. Add **recommendations/suggestions** for stakeholders.
9. State the **exit criteria** (all planned cases run, critical issues closed, open issues planned for next cycle).
10. **Sign off** — whether the test team approves going live.

14.2 Test Reporting Frameworks

A **test framework** is a set of **guidelines** (coding standards, test-data handling, object repository) that make testing more efficient — guidelines, not mandatory rules. Benefits: better efficiency, lower maintenance, minimal manual work, maximum coverage, code reuse, recovery handling, easy reporting.

Common framework types: - **Linear (record & playback)** — independent scripts from recorded actions. - **Module-based** — test the app in smaller parts, then combine for system tests. - **Library-based** — reusable libraries; good for maintenance and scalability. - **Behavior-Driven (BDD)** — tests written in plain, descriptive language so business stakeholders understand them. - **Data-Driven** — same test run with different input data read from a DB/spreadsheet; data separated from scripts. - **Keyword-Driven** — an extension of data-driven; test steps are a sequence of keywords mapped to actions in a shared object repository. - **Hybrid** — combines two or more of the above to get their benefits.

14.3 Software tools for test reporting

- **What they are:** software providing reporting, decision-making, and BI capabilities — presenting data as charts, tables, and dashboards.
- **Why they're needed:** detect bugs early, make analysis easier, and track a product's progress.
- **How to choose one:** **User interface** (clean/easy), **Usability** (easy to learn; compatible with many automation tools), **Functionality/features** (comprehensive relevant reports), and **Value** (price vs features).
- **Key features of good tools:** test-case management, test-execution management, defect management, **traceability**, customizable reports, **dashboards** (release readiness, pass/fail trends), real-time data tracking, and classifying failures by root cause.

-
- **7 testing principles** (Section 1) — the single most likely essay.
 - **QA vs QC vs Testing vs Debugging; Error → Fault → Failure** (17→8→3 example).
 - **Static vs Dynamic = verification vs validation** table.
 - **Review types** comparison (informal / walkthrough / technical / inspection).
 - **EP & BVA** worked examples; **Decision table** = 2^n rules; **State transition** = 4 parts.
 - **Statement vs Decision coverage** (decision $\supseteq$ statement); coverage formula.
 - **4 test levels** + V-model pairings; **Stub vs Driver**.
 - Acceptance forms: **UAT, OAT, CAT, RAT, Alpha, Beta**.
 - **MTBF = MTTF + MTTR**.
 - **STLC 6 phases**; RTM/scope/automation analysis; **Priority (QA) vs Severity (Dev)**; defect life-cycle states.
 - QC: **5 metric categories**, **PCE**, Crosby **COPQ**, **IEEE 829** summary report.
 - **Absolute (/) vs Relative (//) XPath**; XPath axes; `contains()/starts-with()/text()`.
 - Selenium **4 components** (RC deprecated); Grid **Hub-Nodes**; wait types (implicit/explicit/fluent); alert types.
 - **CI vs CD**; Jenkins **Master-Slave**, Jenkinsfile (Declarative vs Scripted), stages Build/Test/Deploy; **Git** = distributed (Torvalds 2005).
 - **Appium**: Node.js server, JSON-over-HTTP, session ID, UIAutomator/XCUITest; native/browser/hybrid apps; 3-layer mobile architecture.

Good luck! 🍀